Frobenius Normalization Enables Stable Training for Quantum State Denoising

Amitav Krishna

***Abstract*— Quantum computers offer asymptotic speedups for problems like molecular simulation and integer factorization, but noise severely limits their near-term utility. Quantum Error Correction can address this, but the number of extra qubits required makes it infeasible for near-term use while the cost per qubit remains high. Quantum Error Mitigation is a near-term alternative, but only recovers expectation values, not the full quantum state needed for extracting accurate results from simulations run on quantum hardware. For these, we need quantum state reconstruction. Quantum state tomography estimates a state by measuring the system across many bases, but these measurements are inherently noisy; quantum state reconstruction recovers a clean density matrix from this noisy data. Neural networks have shown promise here, but previous work has only been demonstrated up to 5 qubits. We find that Frobenius normalization (scaling inputs to unit purity) removes this bottleneck, enabling scaling to 8-qubit systems while maintaining a relatively large improvement in fidelity.**

I. Introduction

Quantum computers hold the potential to enable exponential speedups of many tasks through their exploitation of the principles of quantum mechanics [5], specifically the ability to explore possibilities in superposition, with paths that interfere to amplify correct answers and cancel wrong ones. However, this interference is both a blessing and a curse. After enough time, the computer will inevitably become fully entangled with the environment, locking the information in correlations with the rest of the universe and throwing away the key, leaving us with nothing to work with.

Quantum Error Correction addresses this by encoding a single logical qubit on multiple physical qubits, and then using ancillary qubits to detect errors. To detect errors, we check whether pairs of data qubits match, entangling the ancilla with just this parity information. Crucially, we do not entangle the ancilla with the data itself, so measuring the ancilla only collapses the parity, not the data itself, which stays in superposition. Measuring the bitstring of the ancilla gives us the syndrome, which is then decoded on a classical computer to find the necessary correction. While QEC is a scalable, principled method to reducing errors in quantum computers, useful error rates require on the order of ~1,000+ physical qubits per logical qubit [1], making it infeasible for the near-term [12].

Quantum Error Mitigation, on the other hand, is a method to reducing quantum noise in the short term [3]. QEM requires no extra qubits, and instead runs corrections classically. Common techniques include zero-noise extrapolation, where one runs a circuit multiple times with artificially increased noise levels and extrapolates back to zero noise, and probabilistic error cancellation, which inserts random operations that statistically average out the noise. However, QEM only recovers expectation values—single numbers like $\langle Z \rangle$—not the full quantum state. For tasks that require the complete density matrix, such as verifying entanglement or extracting detailed simulation results, expectation values are not enough.

Quantum State Reconstruction addresses this gap. Like QEM, it uses classical post-processing and requires no extra qubits. But instead of recovering expectation values, QSR recovers the full density matrix from noisy tomographic data. Recent work has shown that neural networks can learn to invert noise channels, taking a noisy density matrix and outputting a denoised estimate [11, 8, 14]. However, previous neural network approaches have only been demonstrated up to 5 qubits. We introduce a data normalization method that enables scaling to 8-qubit systems in our experiments.

We generate synthetic datasets by simulating random quantum circuits composed of single-qubit and two-qubit entangling gates. Each circuit is simulated twice: once exactly to produce a clean density matrix, and once with noise channels (depolarizing, amplitude damping, phase damping, bit flip) applied after each layer to produce a noisy counterpart. We train neural networks to map noisy inputs to clean targets, using Frobenius fidelity as the loss function. The key preprocessing step is Frobenius normalization: we scale each density matrix to unit norm before training. The Frobenius norm of a density matrix equals the square root of its purity—a measure of how "purely quantum" vs. "classically mixed" the state is. Since noisy states have norms up to ~16 times smaller than clean states for 8-qubit systems (a physical consequence of decoherence pushing states toward the maximally mixed state), this normalization decouples scale recovery from structural denoising, allowing the model to focus on restoring coherence structure rather than learning a global rescaling.

On 5-qubit systems, normalization reduces validation loss by 25% and enables both MLP and Transformer architectures to achieve >3 times improvement in Uhlmann fidelity over the noisy baseline. Without normalization, scaling model capacity actually hurts performance—a signature of optimization pathology. On 8-qubit systems (256×256 density matrices), training without normalization fails entirely, while normalized models achieve 3–4 times improvement over baseline. Our results suggest that for neural QSR, input conditioning matters more than architectural choice, as both

Correspondence: akrishna0511@bhncdsb.ca

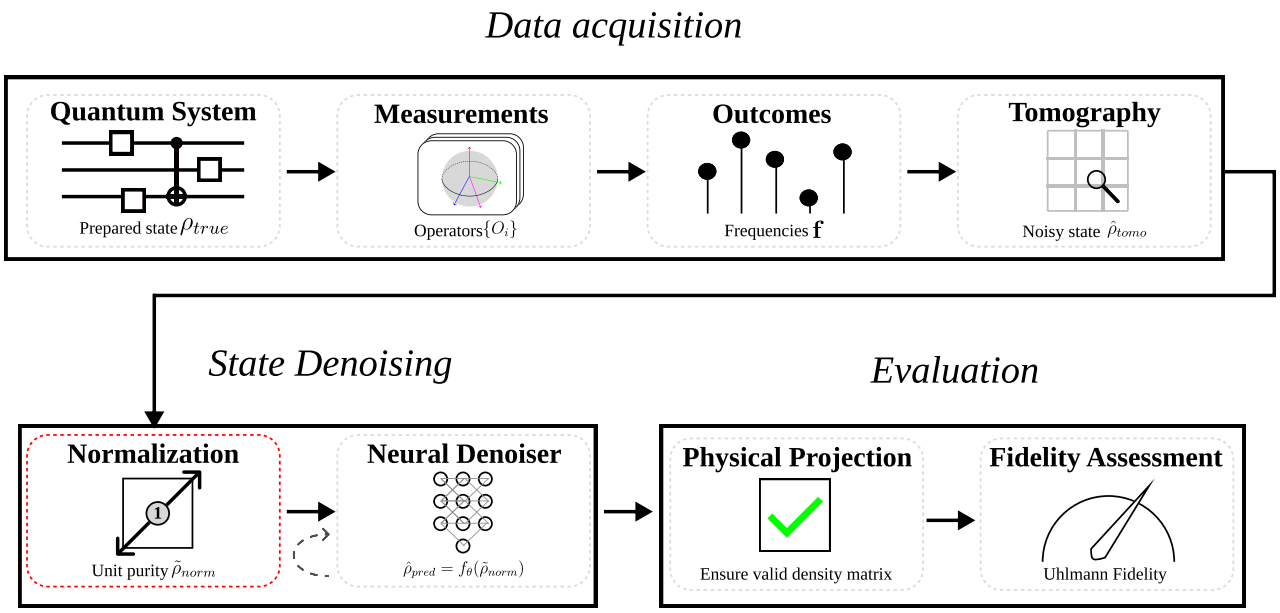


Fig. 1: Overview of the training and evaluation pipeline. Quantum states are measured, reconstructed via tomography, and normalized to unit Frobenius norm. A neural denoiser maps the normalized noisy state to a denoised estimate, which is then projected to a valid density matrix before computing Uhlmann fidelity.

MLPs and Transformers achieve similar performance once properly normalized. The scale mismatch caused by decoherence is a domain-specific challenge that requires domain-specific preprocessing, and we recommend Frobenius normalization as a standard step for any neural approach to quantum state reconstruction.

Section 2 reviews related work on input normalization and neural approaches to quantum error correction. Section 3 describes our dataset generation, model architectures, and the Frobenius normalization method. Section 4 presents our experimental results. Section 5 discusses the physical interpretation and limitations of our approach. Section 6 concludes.

II. Related Work

Input normalization is well-established in deep learning. Batch normalization [6] and layer normalization [2] address internal covariate shift, while input standardization (zero mean, unit variance) is standard practice for feedforward networks. However, quantum density matrices present a unique challenge: the inherent scale difference between noisy and clean states is not statistical variation but a *physical consequence* of decoherence. Our work identifies Frobenius normalization as the appropriate preprocessing for this domain, demonstrating that it is essential for stable training at larger scales.

For computational tractability at larger qubit counts, we adopt patch-based tokenization inspired by Vision Transformers (ViT) [4] and hierarchical variants like the Swin Transformer [10]. We maintain a fixed token count (64) by adjusting patch size with matrix dimension, enabling scaling to 8-qubit systems where element-wise processing is infeasible.

Machine learning has been widely applied to quantum information theory, including for syndrome decoding in quantum error correction [15]. Recent work by Lin et al. [9] has shown success in using convolutional neural networks for classical read-out error mitigation. Unlike traditional read-out error mitigation (REM), which assumes a linear noise map M and attempts to find its inverse M^{-1}, Lin et al. takes the novel approach of assuming that the noise map is instead *non-linear*, and taking a data-driven approach, using a neural network to invert that non-linear noise. Data-driven approaches to quantum state reconstruction have been explored for small numbers of qubits. Morgillo et al.[11] uses a multi-layer perceptron to denoise systems of up to 3 qubits, while Kendre[8] instead uses a convolutional architecture to scale up to 5 qubits.

III. Methods

To investigate the effectiveness of Frobenius normalization for improving neural state denoisers, we generate two datasets consisting of temporally-independent measurement probabilities and coherence values derived from quantum circuits. We generate 100,000 samples for each dataset, each of which consists of a pair of density matrices from the same circuit under ideal and noisy evolution, which we denote ρ_{true} and $\hat{\rho}_{\text{tomo}}$, respectively. Each circuit has a depth uniformly sampled from 6 to 9 (inclusive), and each layer consists of applying a independently uniformly sampled single-qubit gate to every qubit, as well as random two-qubit gates applied to each of $\lfloor n/2 \rfloor$ randomly chosen adjacent pairs.

We consider four standard single-qubit Markovian channels: depolarizing, amplitude damping, phase damping, bit and bit flip, each applied independently to each circuit layer

with strength p. Additionally, we have a mixed channel, which consists of those four channels applied sequentially, each with strength $p/4$. We generate datasets for four noise intensities $p \in \{0.05, 0.10, 0.15, 0.20\}$. The result is a dataset stratified into 20 "noise cells" (5 types $\times$ 4 levels), with equal representation. For both the 5 qubit and 8 qubit datasets, we use 100,000 samples. For both datasets we use an 80/10/10 split for training, validation, and testing stratified by noise cell to ensure balanced evaluation across all noise types and intensities.

We normalize to unit Frobenius norm, a standard measure of matrix scale, prior to training.

$$\tilde{\rho}_{\text{norm}} = \frac{\hat{\rho}_{\text{tomo}}}{\|\hat{\rho}_{\text{tomo}}\|_F}, \quad \|\rho\|_F = \sqrt{\sum_{ij} |\rho_{ij}|^2} \tag{1}$$

Because density matrices are Hermitian ($\rho^\dagger = \rho$), the Frobenius norm reduces to the square root of the purity: $\|\rho\|_F = \sqrt{\text{Tr}(\rho^\dagger \rho)} = \sqrt{\text{Tr}(\rho^2)}$. Since our clean target states are always pure, Frobenius normalization leaves them unchanged ($\|\rho_{\text{true}}\|_F = 1$ already). Normalization only affects the noisy inputs, whose Frobenius norms are suppressed by decoherence: approximately 0.18 for 5-qubit and 0.06 for 8-qubit systems. This discards purity information, which is acceptable since the purity of the clean state is known *a priori*. The practical effect is to bring all inputs to unit scale, which we demonstrate is critical to stable optimization.

For training, we minimize $1 - F_F$, where F_F is the normalized Frobenius fidelity:

$$F_F(\rho, \sigma) = \frac{\text{Tr}(\rho\sigma)}{\sqrt{\text{Tr}(\rho^2)\text{Tr}(\sigma^2)}} \tag{2}$$

For evaluation, we use the Uhlmann fidelity:

$$F_U(\rho, \sigma) = \left(\text{Tr}\sqrt{\sqrt{\rho}\,\sigma\,\sqrt{\rho}}\right)^2 \tag{3}$$

The Frobenius fidelity includes coherence terms algebraically, but it does not model their physical significance, whereas the Uhlmann fidelity does. On the other hand, Uhlmann fidelity requires multiple eigendecompositions, making it extremely expensive to run for training, so we run it only for evaluation. To validate Frobenius fidelity as a training proxy, we analyzed its correlation with the ground-truth Uhlmann fidelity on 100 randomly sampled 5-qubit density matrices (Figure 2). The strong correlation ($r = 0.91$) justifies this choice.

To understand why normalization helps, consider a neural network f_θ trained to minimize a loss $\mathcal{L}(f_\theta(x), y)$. Our training loss F_F is scale-invariant (it is a cosine similarity), so the loss surface is identical regardless of input magnitude. The benefit of normalization is therefore not about what the loss measures, but about how the optimizer navigates that surface.

Consider the first layer of f_θ, which computes $h = W^{(1)}x + b^{(1)}$. The gradient of the loss with respect to these weights is:

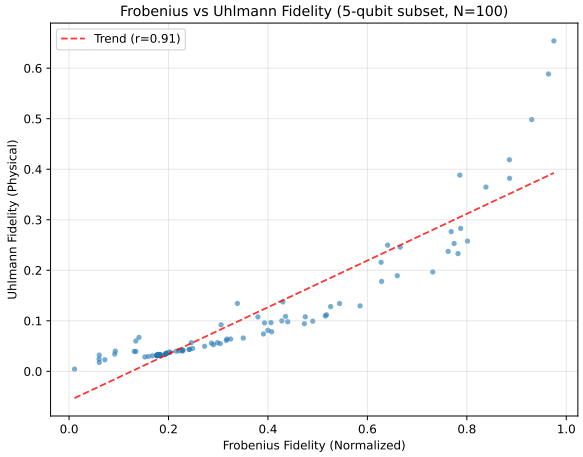


Fig. 2: Correlation between Frobenius fidelity (normalized) and Uhlmann fidelity (physical) on a random subset of the 5-qubit dataset ($r = 0.91$).

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial h} \cdot x^T \tag{4}$$

This gradient is *linearly proportional* to the input x. When inputs are noisy density matrices with $\|x\|_F \approx 0.06$ (8-qubit systems), the gradient magnitude $\|\partial\mathcal{L}/\partial W^{(1)}\|$ is suppressed by the same factor relative to unit-scale inputs. With a fixed learning rate η, the effective parameter update $\Delta W^{(1)} = -\eta\, \partial\mathcal{L}/\partial W^{(1)}$ is ~16$\times$ smaller than it would be for normalized inputs. This is equivalent to an implicit ~16$\times$ reduction in learning rate for the first layer.

The problem is compounded by weight initialization. Our models use PyTorch's default Kaiming uniform initialization, which is calibrated to maintain unit-scale activations and gradients under the assumption that inputs have roughly unit variance. When inputs are 16$\times$ smaller, the initialization places the network in a regime it was not designed for: activations are suppressed from the first layer onward, and the gradient signal attenuates further as it propagates through subsequent layers.

Critically, this effect worsens exponentially with system size. The Frobenius norm of the maximally mixed state is $1/\sqrt{2^n}$, so gradient suppression scales as $O(2^{-n/2})$. This explains why normalization is merely helpful at 5 qubits (~5 times suppression) but essential at 8 qubits (~16 $\times$ suppression). By normalizing inputs to unit Frobenius norm, we restore gradient magnitudes and initialization assumptions to their intended operating regime.

Note that Frobenius-normalized matrices are not physically valid density matrices as their trace is not equal to one. We use them only as an intermediate representation during training. At evaluation time, we project the denoiser output $\hat{\rho}_{\text{pred}} = f_\theta(\tilde{\rho}_{\text{norm}})$ back to a valid density matrix by Hermitianizing, normalizing the trace, and clamping negative eigenvalues:

$$\frac{\hat{\rho}_{\text{pred}} + \hat{\rho}_{\text{pred}}^\dagger}{2} \to \rho_{\text{herm}} \tag{5}$$

$$\frac{\rho_{\text{herm}}}{\text{Tr}(\rho_{\text{herm}})} \to \rho_{\text{proj}} \tag{6}$$

$$\max(0,\, \lambda_i) \to \lambda_i, \quad V\,\text{diag}(\lambda_{\text{clamped}})\,V^\dagger \to \rho_{\text{psd}} \tag{7}$$

where λ_i and V are the eigenvalues and eigenvectors of ρ_{proj}, respectively. Note that clamping can slightly reduce the trace (by the magnitude of the clamped eigenvalues, typically 10^{-7}); we do not re-normalize afterward, as the effect on fidelity is negligible. This projection prevents evaluation from returning nonsensical values, such as fidelities greater than one.

Both architectures share a common patch tokenization scheme. The input density matrix is divided into non-overlapping patches, each embedded via a linear projection into 64 tokens of dimension 128 (5-qubit: 4×4 patches; 8-qubit: 32×32 patches). Each decoder token is projected back to its corresponding patch via a linear layer, then patches are reassembled into the full matrix.

The Transformer uses an encoder-decoder structure: 4 encoder layers and 4 decoder layers, each with 8 attention heads and FFN dimension 512. The decoder uses cross-attention to the encoder. The MLP uses an MLP-Mixer-style architecture [13]: token mixing (hidden dim 384) that mixes across patches, and channel mixing (hidden dim 320) applied per-patch. Parameter counts are matched: $1.09\,\text{M}$ (5-qubit) and $1.61\,\text{M}$ (8-qubit), with the difference coming from patch embedding/unembedding layers that scale with patch size.

To test whether larger models benefit more from normalization, we trained two additional MLP variants with identical optimization hyperparameters. The "Wide" variant scales hidden dimensions by 4 times (token hidden 1536, channel hidden 1280), yielding $5.29\,\text{M}$ parameters, nearly five times the baseline. The "Deep" variant doubles the layer count (8+8 encoder-decoder blocks), yielding $2.15\,\text{M}$ parameters.

All models are trained with AdamW (lr 3×10^{-4}, weight decay 10^{-5}), batch size 256, for up to 100 epochs with early stopping (patience 15 on validation loss). The training loss is $1 - F_F$ (Frobenius fidelity).

IV. Results and Discussion

Our central finding is that Frobenius normalization dramatically improves training stability and final model performance. Figure 3 validation loss with and without normalization on the 5-qubit dataset.

Figure 3 illustrates the training dynamics: the normalized MLP converges faster and to a substantially lower loss, while the unnormalized MLP plateaus early and exhibits greater instability.

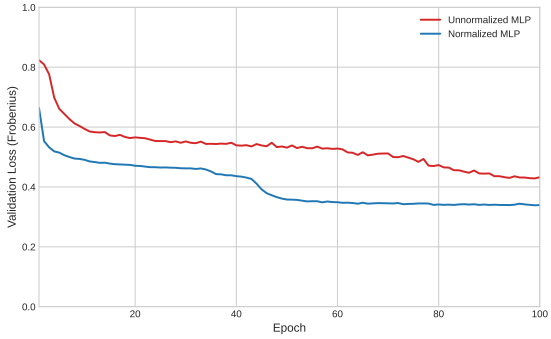


Fig. 3: Validation loss curves for the 5-qubit MLP with and without Frobenius normalization. Normalization enables faster convergence and a 25% lower final loss.

Without normalization, the small input magnitudes (~0.18 for 5 qubits, ~0.06 for 8 qubits) suppress gradient flow through early layers, as described in the Methods section. This gradient suppression prevents the model from learning fine-grained coherence recovery. With Frobenius normalization, both architectures achieve substantial improvements over the noisy baseline:

TABLE I: 5-qubit model performance with Frobenius normalization. Both models use identical patch tokenization (4×4 patches, 64 tokens) and matched parameter counts.

Model	Params	Uhlmann Fidelity	vs Baseline
Baseline (noisy input)	-	0.167	1.00 times
MLP (hierarchical)	1.09M	0.516	3.09 times
Transformer (hierarchical)	1.09M	0.525	3.14 times

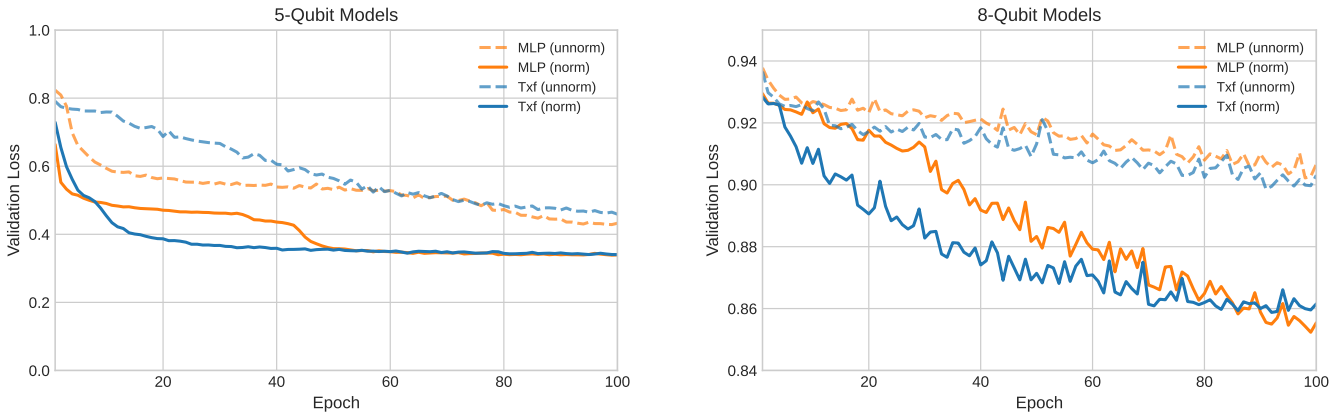
Comparing to the unnormalized MLP (0.463 Uhlmann fidelity), normalization improved the MLP from 0.463 to 0.516 (+11%) and the Transformer from 0.420 to 0.525 (+25%). The Transformer's larger gain likely reflects a conflict between its internal LayerNorm layer which run on the assumption that inputs have a consistent scale across the token sequence and the extremely variable magnitudes of the unnormalized data. Both architectures achieve similar performance with proper normalization, confirming that input conditioning and not architectural choiceis the critical factor for this task.

To confirm that the performance ceiling without normalization is an optimization problem rather than a capacity limitation, we trained wider (4× hidden dims, 5.29M params) and deeper (2× layers, 2.15M params) MLP variants under identical unnormalized conditions:

TABLE II: Capacity ablations on unnormalized 5-qubit data. Larger models perform worse, confirming an optimization pathology.

Model	Params	Uhlmann Fidelity	vs Baseline
MLP (matched)	1.09M	0.463	2.77 times
MLP (wide)	5.29M	0.310	1.86 times
MLP (deep)	2.15M	0.407	2.44 times

Both larger models perform worse, with the wide MLP dropping 33% (0.310 vs 0.463) and the deep MLP dropping



(a) 5-qubit models

(b) 8-qubit models (y-axis truncated)

Fig. 4: Validation loss curves for unnormalized (dashed) and normalized (solid) models. (a) 5-qubit models show a clear separation between normalized and unnormalized training. (b) 8-qubit models reveal that normalized models steadily improve while unnormalized models plateau.

12% (0.407 vs 0.463). If the performance ceiling were due to insufficient capacity, adding parameters should help, not hurt. The fact that it hurts confirms that the bottleneck is optimization, not capacity.

Adding normalization to these same models confirms this diagnosis selectively: the matched MLP improves from 0.463 to 0.516 (+11%), but the wide MLP reaches only 0.344 and the deep MLP only 0.349—both far below the matched model. Normalization resolves optimization pathologies that arise from small input magnitudes (Section 3), but it cannot resolve optimization pathologies that arise from other sources.

The most dramatic demonstration of normalization's importance is at 8 qubits. Previous attempts to train on 8-qubit density matrices (256×256, 65,536 complex elements) without normalization failed completely—models produced outputs no better than the noisy input. With normalization, both architectures achieve significant improvement:

TABLE III: 8-qubit scaling results with Frobenius normalization. Despite the exponentially harder task (baseline 0.006), both models achieve more than 3 times improvement.

Model	Uhlmann Fidelity	vs Baseline
Baseline (noisy)	0.00635	1.00 times
MLP (hierarchical)	0.02385	3.76 times
Transformer (hierarchical)	0.01901	3.00 times

The 8-qubit task is exponentially harder: the baseline fidelity drops from 0.167 to 0.006, and the density matrix has 64 times more elements. Yet with normalization, the MLP achieves 3.76 times improvement over baseline—comparable relative improvement to the 5-qubit case. Unnormalized 8-qubit models failed to converge (Figure 4b), so we do not report Uhlmann fidelity for these models.

validation loss for all four model/qubit combinations with and without normalization. The 5-qubit normalized curves show rapid early improvement followed by gradual refinement, while their unnormalized counterparts converge more slowly to a higher loss. The 8-qubit contrast is starker: although all four 8-qubit curves are noisy, by epoch 30 the normalized models have decisively overtaken the unnormalized models, which plateau near 0.91 while normalized models steadily decrease to ~0.85. To verify that stable convergence is not seed-dependent, we retrained 5-qubit models with two additional initialization seeds (100 and 200) while keeping the data split fixed (seed=42). All runs converged to similar validation loss (0.32–0.35), confirming reproducibility.

Init Seed	MLP Val Loss	Transformer Val Loss
42	0.340	0.328
100	0.337	0.347
200	0.323	0.345

For computational tractability at larger qubit counts, we use patch-based tokenization that maintains a fixed token count (64) regardless of system size, trading spatial resolution for scalability (5-qubit: 4×4 patches, 32 complex values per patch; 8-qubit: 32×32 patches, 2048 complex values per patch). The 8-qubit results validate this approach: despite extreme compression (2048:1), both models achieve more than 3 times baseline improvement.

Our 8-qubit results also expose a fundamental limitation of hierarchical tokenization: when patches become too large, the amount of information decreases tremendously. This approach requires patches small enough to preserve local structure while remaining computationally tractable. As the size of a density matrix is exponential to the number of qubits, future work will have to explore methods of balancing compression and tractability.

Our synthetic noise model assumes independent, Markovian noise channels acting on each qubit (depolarizing, amplitude damping, phase damping). Real quantum processors exhibit significant crosstalk (correlated errors between qubits) and non-Markovian memory effects. Extending our normalization approach to models trained on realistic correlated noise is an important direction for future work.

All experiments in this work rely on classical simulation of quantum states and noise. Deploying these models on actual hardware introduces challenges such as calibration drift, where the noise characteristics of the device change

over time. A model trained on a static distribution of noise levels may degrade as the hardware drifts. Future work will investigate strategies for online adaptation or robust training across a wider envelope of noise parameters to mitigate this issue.

Our current approach enforces physical validity (Hermiticity, positive semi-definiteness, unit trace) only at evaluation time via post-hoc projection. An alternative would be to enforce these constraints on intermediate representations during training, potentially providing stronger inductive bias. Earlier experiments with Cholesky-constrained outputs which guarantee physically valid outputs resulted in model collapse due to a non-convex optimization surface. Future will explore methods of ensuring physicality in outputs and the effects of enforcing physicality for intermediate representations on the performance of the models.

V. Conclusion and Future Directions

Frobenius normalization has been applied to the preprocessing of quantum state data. It is observed that multilayer perceptrons and transformers trained with this normalized data scale better when compared to their counterparts trained with unnormalized data. In this work, every model must learn a mapping for every noise channel silmultaneously. Previous work by Morgillo et al. [11] has demonstrated the effectiveness of neural networks for the classification of different kinds of noise channels. Additional improvements may come from having an architecture with multiple noise-channel specific neural networks paired with a noise-channel classifier that functions as a router, analagous to a model of experts architecture [7]. Other areas of future work will include scaling up the datasets to larger numbers of qubits.

VI. Acknowledgements

The author used a large language model to edit parts of this work for clarity in presentation, and for debugging failed experiments and assisting with the code implementation. The technical content, experimental design, and interpretations are the author's own, and the author takes full responsibility for this work.

References

[1] Rajeev Acharya et al. "Quantum error correction below the surface code threshold". In: *Nature* 638.8052 (Dec. 2024), pp. 920–926. ISSN: 1476-4687. DOI: 10.1038/s41586-024-08449-y. URL: http://dx.doi.org/10.1038/s41586-024-08449-y.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. "Layer Normalization". In: *arXiv preprint arXiv:1607.06450* (2016).

[3] Zhenyu Cai et al. "Quantum error mitigation". In: *Rev. Mod. Phys.* 95 (4 Dec. 2023), p. 045005. DOI: 10.1103/RevModPhys.95.045005. URL: https://link.aps.org/doi/10.1103/RevModPhys.95.045005.

[4] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". In: *arXiv preprint arXiv:2010.11929* (2020).

[5] Richard P. Feynman. "Simulating physics with computers". In: *International Journal of Theoretical Physics* 21.6-7 (1982), pp. 467–488.

[6] Sergey Ioffe and Christian Szegedy. "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift". In: *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. PMLR. 2015, pp. 448–456.

[7] Robert A. Jacobs et al. "Adaptive Mixtures of Local Experts". In: *Neural Computation* 3.1 (1991), pp. 79–87.

[8] Karan Kendre. *Machine Learning for Quantum Noise Reduction*. 2025. arXiv: 2509.16242 [quant-ph]. URL: https://arxiv.org/abs/2509.16242.

[9] Xiao-Dao Lin et al. "Quantum Error Mitigation via Autoencoder Neural Networks". In: *2025 IEEE International Conference on Quantum Control, Computing and Learning (qCCL)*. June 2025, pp. 58–64. DOI: 10.1109/qCCL65142.2025.11158291. (Visited on 01/22/2026).

[10] Ze Liu et al. "Swin Transformer: Hierarchical Vision Transformer using Shifted Windows". In: *arXiv preprint arXiv:2103.14030* (2021).

[11] Angela Rosy Morgillo et al. "Quantum state reconstruction in a noisy environment via deep learning". In: *Quantum Machine Intelligence* 6.2 (July 2024). ISSN: 2524-4914. DOI: 10.1007/s42484-024-00168-x. URL: http://dx.doi.org/10.1007/s42484-024-00168-x.

[12] John Preskill. "Quantum Computing in the NISQ era and beyond". In: *Quantum* 2 (2018), p. 79.

[13] Ilya O. Tolstikhin et al. "MLP-Mixer: An all-MLP Architecture for Vision". In: *Advances in Neural Information Processing Systems* 34 (2021), pp. 24261–24272.

[14] Giacomo Torlai et al. "Neural-network quantum state tomography". In: *Nature Physics* 14.5 (Feb. 2018), pp. 447–450. ISSN: 1745-2481. DOI: 10.1038/s41567-018-0048-5. URL: http://dx.doi.org/10.1038/s41567-018-0048-5.

[15] Hanrui Wang et al. *Transformer-QEC: Quantum Error Correction Code Decoding with Transferable Transformers*. 2023. arXiv: 2311.16082 [quant-ph]. URL: https://arxiv.org/abs/2311.16082.